Name : M.AbuBakar
Roll No: **BSCS25109**

OOP PROJECT

QCrypt

Classical & Quantum Cryptography Engine

Object Oriented Programming

Inheritance & Polymorphism

Implemented Ciphers:

- Caesar
- ROT13
- Vigenere
- XOR
- RSA
- BB84 Quantum

QCrypt -- Classical & Quantum Cryptography Engine

OOP C++ Project Documentation

Project Overview

QCrypt is a comprehensive cryptography engine built in C++ using Object-Oriented Programming. It implements five classical cipher algorithms and one quantum key distribution protocol (BB84), all unified under a single abstract base class hierarchy.

The system encrypts and decrypts user-provided text using six different algorithms -- Caesar, ROT13, Vigenere, XOR, RSA, and BB84 Quantum -- demonstrating inheritance and runtime polymorphism throughout.

Functional Requirements

- Accept plaintext input from the user
- Encrypt the input using all six cipher algorithms
- Decrypt each ciphertext back to the original message
- Verify correctness by comparing decrypted output with original input
- Simulate BB84 Quantum Key Distribution with eavesdrop detection
- Support three quantum channel types: Ideal, Noisy, and Eavesdropped
- Manage all ciphers through a unified polymorphic interface

OOP Concepts Implemented

2.1 Inheritance

Inheritance Type	Example
Single Inheritance	CaesarCipher inherits from SubstitutionCipher
Multi-level Inheritance	BB84Cipher -> QuantumCipher -> Cipher
Hierarchical Inheritance	CaesarCipher and ROT13Cipher both inherit
Abstract Base Classes	Cipher, MathEngine, KeyGenerator, Basis,QuantumChannel, Party

2.2 Polymorphism

Pure Virtual Functions	encrypt(), decrypt(), showInfo(), getName(),compute(),measure(), transmit(), prepare()
Runtime Polymorphism	CryptoManager calls encrypt() on Cipher* without knowing derived type
Dynamic Dispatch	channel->transmit() resolves toIdealChannel, NoisyChannel, or EavesdroppedChannel at runtime
Virtual Destructors	virtual ~Cipher() ensures correct cleanupthrough base pointers

2.3 Encapsulation

All cipher keys, internal state, and private data members are declared as private or protected. Public interfaces expose only the encrypt(), decrypt(), and showInfo() methods.

Complete Class Hierarchy:

```
Cipher                          (abstract base - ALL ciphers)
|
+-- SubstitutionCipher          (abstract - character shifting)
|   +-- CaesarCipher            shifts each letter by a fixed number
|   +-- ROT13Cipher             fixed shift of 13, self-inverse
|
+-- PolyalphabeticCipher        (abstract - keyword-based shifting)
|   +-- VigenereCipher          different shift per character using keyword
|
+-- BitwiseCipher               (abstract - bit-level operations)
|   +-- XORCipher               XOR each byte with a key
|
+-- AsymmetricCipher            (abstract - public/private key system)
|   +-- RSACipher               prime numbers + modular arithmetic
|
+-- QuantumCipher               (abstract - quantum key distribution)
    +-- BB84Cipher              BB84 protocol + derived XOR encryption


MathEngine                      (abstract base - mathematical operations)
+-- ModularArithmetic           fast exponentiation, GCD, modular inverse
+-- PrimeEngine                 primality testing, prime generation


KeyGenerator                    (abstract base - key creation)
+-- PrimeKeyGenerator           generates RSA key pair (p, q, e, d)
+-- SimpleKeyGenerator          generates shift value / keyword


Basis                           (abstract base - quantum measurement)
+-- RectilinearBasis (+)        measures 0 deg (bit 0) and 90 deg (bit 1)
+-- DiagonalBasis    (x)        measures 45 deg (bit 0) and 135 deg (bit 1)


QuantumChannel                  (abstract base - photon transmission)
+-- IdealChannel                photon arrives unchanged, no Eve
+-- NoisyChannel                10% random photon disturbance
+-- EavesdroppedChannel         Eve intercepts - causes 25% error rate


Party                           (abstract base - protocol participants)
+-- Alice                       sender - prepares and sends photons
+-- Bob                         receiver - measures incoming photons
+-- Eve                         eavesdropper - intercepts in channel


CryptoManager                   manages all ciphers, runs them together
```

Total Classes: 22 | Abstract Classes: 7 | Max Inheritance Depth: 3

Module Breakdown

Module 1 - Math Engine

Class	Responsibility
MathEngine	Abstract base with pure virtual compute(a, b, mod)
ModularArithmetic	Fast modular exponentiation, GCD (Euclidean), Modular Inverse (Extended Euclidean)
PrimeEngine	Trial division primality test, random prime generation

Module 2 - Key Generator

Class	Responsibility
KeyGenerator	Abstract base with generate() and show()
PrimeKeyGenerator	Generates RSA key pair: finds p, q, computes n, phi(n), e,d
SimpleKeyGenerator	Stores a shift number and/or keyword string

Module 3 - Classical Ciphers

Class	Algorithm
CaesarCipher	Shift each letter by key positions in alphabet
ROT13Cipher	Fixed shift of 13 - encoding and decoding use same function
VigenereCipher	Shift each letter by corresponding keyword letter
XORCipher	XOR each byte with the key byte
RSACipher	Encrypt: C = M^e mod n, Decrypt: M = C^d mod n

Module 4 - Quantum Layer (BB84)

Class	Responsibility
Basis	Abstract measurement framework
RectilinearBasis	Correctly measures 0/90 degree photons
DiagonalBasis	Correctly measures 45/135 degree photons
QuantumChannel	Abstract photon transmission medium
IdealChannel	Passes photon unchanged
NoisyChannel	10% random disturbance
EavesdroppedChannel	Eve randomly disturbs photons - creates detectable errors
Alice	Prepares random bits and random bases, encodes as polariz
Bob	Picks random bases, measures received photons
Eve	Intercepts channel with her own random basis
BB84Cipher	Runs full protocol, extracts shared key, uses it for XOR

Module 5 - Crypto Manager

CryptoManager holds an array of Cipher* base class pointers. The runAll() method loops through this array calling encrypt(), decrypt(), and showInfo() on each pointer -- never knowing which derived class it is calling. This is runtime polymorphism in its most direct form.

Cipher Algorithms

5.1 Caesar Cipher

Shifts each letter forward by key positions in the alphabet.

Formula: C = (M + key) mod 26 | M = (C - key + 26) mod 26

```
Key = 7
H -> O  (7 + 7 = 14 -> O)
E -> L  (4 + 7 = 11 -> L)
HELLO -> OLSSV
Decrypt: shift backward by 7 -> HELLO
```

5.2 ROT13 Cipher

A special Caesar cipher with fixed shift of 13. Since 13+13=26, encoding equals decoding.

```
HELLO -> URYYB
Apply ROT13 again -> HELLO (self-inverse)
```

5.3 Vigenere Cipher

Each letter is shifted by the corresponding letter of a repeating keyword.

```
Message: HELLO WORLD
Keyword:QCRYPT        (repeating)

      H + Q = X   (7+16=23 -> X)
      E + C = G   (4+2=6 -> G)
      L + R = C   (11+17=28 mod 26 = 2 -> C)
```

Caesar because each letter has a different shift -- frequency analysis is ineffective.

5.4 XOR Cipher

Each byte is XOR'd with the key byte. XOR is its own inverse: encrypt = decrypt.

```
Key = 0b10110101 (181)
H = 01001000 XOR 10110101 = 11111101 (hex: FD)
XOR same key again -> 01001000 -> H
```

5.5 RSA-Style Cipher

Uses real number theory. Mathematics are identical to actual RSA with smaller key sizes for educational purposes.

Key Generation:

```
1. Pick two primes:   p = 61,  q = 53
2. Compute:           n = p * q = 3233
3. Euler's totient:   phi(n) = (p-1)(q-1) = 3120
4. Choose public e:   gcd(e, phi(n)) = 1  ->  e = 17
5. Find private d:    d * e = 1 (mod phi(n))  ->  d = 2753
```

Encryption (public key e=17, n=3233): C = 72^17 mod 3233 = 2790

Decryption (private key d=2753, n=3233): M = 2790^2753 mod 3233 = 72 -> H

5.6 BB84 Quantum Cipher

BB84 is a Quantum Key Distribution (QKD) protocol. Its security is guaranteed by the laws of quantum physics: measuring a quantum state disturbs it.

Photon Polarization Encoding:

Basis	Bit 0	Bit 1
Rectilinear	0 deg	90 deg
Diagonal	45 deg	135 deg

Protocol Steps:

Step 1 - Alice sends photons with random bits and random bases.

Step 2 - Bob randomly picks measurement bases.

Step 3 - When bases match, Bob gets the correct bit. When bases differ, Bob gets a random result.

Step 4 - Basis reconciliation: Alice and Bob publicly compare which bases they used (not the bits).

Step 5 - Extract shared secret key from positions where bases matched.

Step 6 - Eavesdrop detection: Compare a sample of key bits. If error rate exceeds 20%, Eve is detected and transmission is aborted.

Step 7 - Use shared key for XOR encryption of the message.

Polymorphism Demonstration

CryptoManager::runAll() - The Core Polymorphic Loop

ciphers[i] is a Cipher* -- an abstract base class pointer. At runtime, it resolves to the correct derived class:

- Slot 0 -> CaesarCipher::encrypt()
- Slot 1 -> ROT13Cipher::encrypt()
- Slot 2 -> VigenereCipher::encrypt()
- Slot 3 -> XORCipher::encrypt()
- Slot 4 -> RSACipher::encrypt()
- Slot 5 -> BB84Cipher::encrypt()

The loop never changes. The behavior changes completely based on the derived type.

Sample Program Output

```
======================================================
       QCrypt - Cryptography Engine
       Classical + Quantum (BB84)
======================================================

  Enter message to encrypt: HELLO

  [1] Caesar Cipher       Encrypted: OLSSV       Decrypted: HELLO [OK]
  [2] ROT13 Cipher        Encrypted: URYYB       Decrypted: HELLO [OK]
  [3] Vigenere Cipher     Encrypted: XGCZB       Decrypted: HELLO [OK]
  [4] XOR Cipher          Encrypted: FD BC A4..  Decrypted: HELLO [OK]
  [5] RSA-style Cipher    Encrypted: [2790][1231] Decrypted: HELLO [OK]
  [6] BB84 Quantum        Key: 11001010          Decrypted: HELLO [OK]
```

Project Summary:

Cipher	Type	Key	Math Used
Caesar	Substitution	shift=7	(M+k) mod 26
ROT13	Substitution	shift=13	(M+13) mod 26
Vigenere	Polyalphabetic	keyword	(M+Ki) mod 26 per char
XOR	Bitwise	byte key	M XOR k
RSA	Asymmetric	(e,d,n)	M^e mod n
BB84	Quantum	photon polarizations	Heisenberg + XOR